

713 小结

模板是 C++ 语言与众不同的特性，也是标准库的基础。一个模板就是一个编译器用来生成特定类类型或函数的蓝图。生成特定类或函数的过程称为实例化。我们只编写一次模板，就可以将其用于多种类型和值，编译器会为每种类型和值进行模板实例化。

我们既可以定义函数模板，也可以定义类模板。标准库算法都是函数模板，标准库容器都是类模板。

显式模板实参允许我们固定一个或多个模板参数的类型或值。对于指定了显式模板实参的模板参数，可以应用正常的类型转换。

一个模板特例化就是一个用户提供的模板实例，它将一个或多个模板参数绑定到特定类型或值上。当我们不能（或不希望）将模板定义用于某些特定类型时，特例化非常有用。

最新 C++ 标准的一个主要部分是可变参数模板。一个可变参数模板可以接受数目和类型可变的参数。可变参数模板允许我们编写像容器的 `emplace` 成员和标准库 `make_shared` 函数这样的函数，实现将实参传递给对象的构造函数。

术语表

类模板 (class template) 模板定义，可从它实例化出特定的类。类模板的定义以关键字 `template` 开始，后跟尖括号对 `<` 和 `>`，其内为一个用逗号分隔的一个或多个模板参数的列表，随后是类的定义。

默认模板实参 (default template argument) 一个类型或一个值，当用户未提供对应模板实参时，模板会使用它。

显式实例化 (explicit instantiation) 一个声明，为所有模板参数提供了显式实参。714 实例 (instantiation) 编译器从模板生成用来指导实例化过程。如果声明是 `extern` 的，模板将不会被实例化；否则，模板将利用指定的实参进行实例化。对每个 `extern` 模板声明，在程序中某处必须有一个非 `extern` 的显式实例化。

显式模板实参 (explicit template argument) 在一个函数调用中或定义模板类类型时，由用户提供的模板实参。显式模板实参在紧跟在模板名的尖括号对中给出。

函数参数包 (function parameter pack) 表示零个或多个函数参数的参数包。

函数模板 (function template) 模板定义，可从它实例化出特定函数。函数模板的定

义以关键字 `template` 开始，后跟尖括号对 `<` 和 `>`，其内为一个用逗号分隔的一个或多个模板参数的列表，随后是函数的定义。

实例化 (instantiate) 编译器处理过程，用实际的模板实参来生成模板的一个特殊实例，其中参数被替换为对应的实参。当函数模板被调用时，会自动根据传递给它的实参来实例化。而使用类模板时，则需要我们提供显式模板实参。

实例 (instantiation) 编译器从模板生成的类或函数。

成员模板 (member template) 本身是模板的成员函数。成员模板不能是虚函数。

非类型参数 (nontype parameter) 表示值的模板参数。非类型模板参数的实参必须是常量表达式。

包扩展 (pack expansion) 处理过程，将一个参数包替换为其中元素的列表。

参数包 (parameter pack) 表示零个或多个参数的模板或函数参数。

部分特例化 (partial specialization) 类模板的一个版本，其中指定了某些但不是所

有模板参数，或是一个或多个参数的属性未被完全指定。

模式 (pattern) 定义了扩展后参数包中每个元素的形式。

模板实参 (template argument) 用来实例化模板参数的类型或值。

模板实参推断 (template argument deduction) 编译器确定实例化哪个函数模板的过程。编译器检查那些使用模板参数的实参的类型，将这些类型或值绑定到模板参数，来自动实例化一个函数版本。

模板参数 (template parameter) 在模板参数列表中指定的名字，可在模板定义内部使用。模板参数可以是类型参数，也可以是非类型参数。为了使用一个类模板，我们必须为每个模板参数提供显式实参。编译器使用这些类型或值实例化出一个类版本，其中所有用到模板参数的地方都被替换为实际的实参。当使用一个函数模板时，编译器使用调用中的函数实参推断模板实参，并使用推断出的模板实参实例化出一个特定的函数。

模板参数列表 (template parameter list) 用逗号分隔的参数列表，用于模板的定义

或声明中。每个参数可以是一个类型参数，也可以是一个非类型参数。

模板参数包 (template parameter pack) 表示零个或多个模板参数的参数包。

模板特例化 (template specialization) 类模板、类模板的成员或函数模板的重定义，其中指定了某些（或全部）模板参数。模板特例化版本必须出现在原模板的声明之后，必须出现在任何利用特殊实参来使用模板的代码之前。一个函数模板中的每个模板参数都必须完全特例化。

类型参数 (type parameter) 模板参数列表中的名字，用来表示类型。类型参数在关键字 `typename` 或 `class` 之后指定。

类型转换 (type transformation) 由标准库定义的类模板，可将给定的模板类型参数转换为一个相关类型。

可变参数模板 (variadic template) 接受可变数目模板实参的模板。模板参数包用省略号指定（如 `class...`、`typename...` 或 `type-name...`）

第IV部分

高级主题

内容

第 17 章 标准库特殊设施.....	635
第 18 章 用于大型程序的工具.....	683
第 19 章 特殊工具与技术.....	725

第IV部分将介绍 C++和标准库的一些附加特性，虽然这些特性在特定的情况下很有用，但并非每个 C++程序员都需要它们。这些特性分为两类：一类对于求解大规模的问题很有用；另一类适用于特殊问题而非通用问题。针对特殊问题的特性既有属于 C++语言的（将在第 19 章介绍），也有属于标准库的（将在第 17 章进行介绍）。

在第 17 章中我们介绍四个具有特殊目的的标准库设施：`bitset` 类和三个新标准库设施（`tuple`、正则表达式和随机数）。我们还将介绍 IO 库中某些不常用的部分。

第 18 章介绍异常处理、命名空间和多重继承。这些特性在设计大型程序时是最有用的。

即使是一个程序员就能编写的足够简单的程序，也能从异常处理机制受益，这也是为什么我们在第 5 章介绍了异常处理的基本知识的原因。但是，对于需要大型团队才能完成的程序设计问题，运行时错误处理才显得更为重要也更难于管理。在第 18 章中，我们会额外介绍一些有用的异常处理设施。我们还将详细讨论异常是如何处理的，并展示如何定义和使用自己的异常类。这一章还会介绍新标准中异常处理方面的改进——如何指出一个特定函数不会抛出异常。

大型应用程序通常会使用来自多个提供商的代码。如果提供商不得不将他们定义的名字放置在单一的命名空间中，那么将多个独立开发的库组合起来是很困难的（如果能组合的话）。独立开发的库几乎必然会使用与其他库相同的名字；对于某个库中定义的名字，如果另一个库中使用了相同的名字，就会引起冲突。为了避免名字冲突，我们可以在一个 `namespace` 中定义名字。

无论何时我们使用一个来自标准库的名字，实际上都是在使用名为 `std` 的命名空间中的名字。第 18 章将会展示如何定义我们自己的命名空间。

第 18 章最后介绍一个很重要但不太常用的语言特性：多重继承。多重继承对非常复杂的继承层次很有用。

第 19 章介绍几种用于特定类别问题的特殊工具和技术，包括如何重定义内存分配机制；C++对运行时类型识别（`run-time type identification`, RTTI）的支持——允许我们在运行时才确定一个表达式的实际类型；以及如何定义和使用指向类成员的指针。类成员指针不同于普通数据或函数指针。普通指针仅根据对象或函数的类型而变化，而类成员指针还必须反映成员所属的类。我们还将介绍三种附加的聚合类型：联合、嵌套类和局部类。这一章最后将简要介绍一组本质上不可移植的语言特性：`volatile` 修饰符、位域以及链接指令。

第 17 章

标准库特殊设施

内容

17.1 tuple 类型	636
17.2 bitset 类型	640
17.3 正则表达式	645
17.4 随机数	659
17.5 IO 库再探	666
小结	680
术语表	680

最新的 C++ 标准极大地扩充了标准库的规模和范围。实际上，从 1998 年的第一版标准到 2011 年的最新标准，标准库部分的篇幅增加了两倍以上。因此，介绍所有 C++ 标准库类的知识大大超出了本书范围。但是，有 4 个标准库设施，虽然它们比我们已经介绍的其他标准库设施更特殊，但也足够通用，应该放在一本入门书籍中进行介绍。这 4 个标准库设施是：tuple、bitset、随机数生成及正则表达式。此外，我们还将介绍 IO 库中一些具有特殊目的的部分。